3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

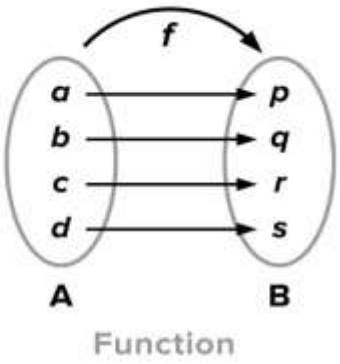
Classes : Offline & Online

Daily Teaching : 2 Hours

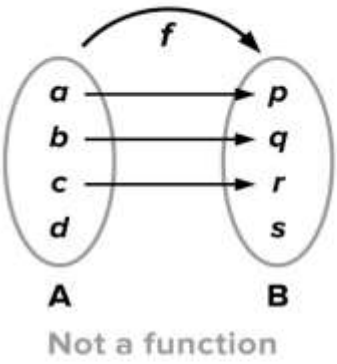
Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

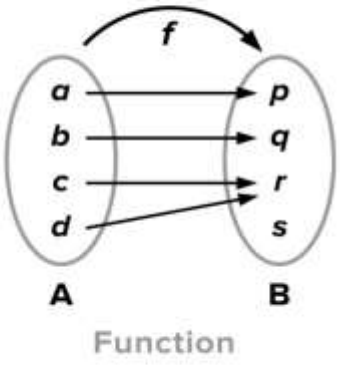
Class 9 – Functions: Advanced Concepts



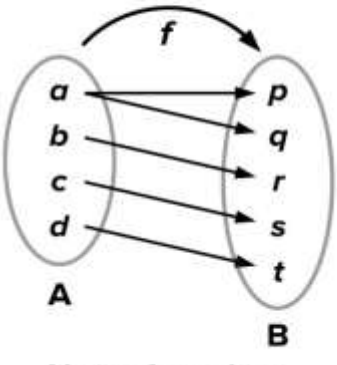
Function



Not a function



Function



Not a function

Learning Objectives

By the end of this class, students will be able to:

- Understand functions and why they are important.
- Create user-defined functions.
- Use parameters and arguments.
- Use return statements.
- Understand local and global variables.
- Use default and keyword arguments.
- Pass multiple values to functions.
- Write reusable functions.
- Create functions that perform calculations and data processing.
- Combine multiple functions into one application.
- Build a **Student Result Analyzer** using reusable functions.

3DWEBSOFT FOUNDATION

1. What is a Function?

A **function** is a reusable block of Python code designed to perform a specific task.

Instead of writing the same code repeatedly, we write it once inside a function and call it whenever needed.

Without a Function

id="j8s1a2"

print("Welcome to Python")

print("Welcome to Python")

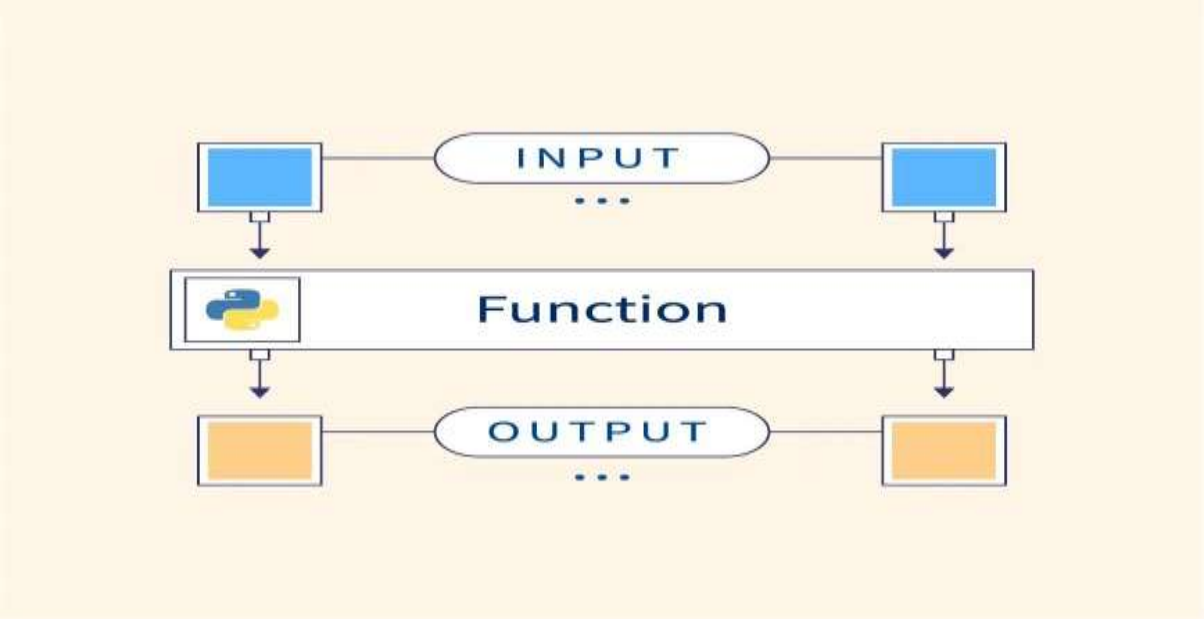
print("Welcome to Python")

With a Function

```
def welcome():
    print("Welcome to Python")


welcome()
welcome()
welcome()
```

The function allows us to **write once and reuse many times**.



2. Why Do We Need Functions?

Imagine a school management program containing:

- Student registration
- Marks calculation
- Grade calculation
- Attendance calculation
- Fee calculation
- Report generation

If we write everything in one large program, the code becomes difficult to understand.

Functions allow us to divide the program into smaller tasks.

Student Management System

```
        |
        ├── add_student()
        ├── calculate_marks()
        ├── calculate_grade()
        ├── check_attendance()
        └── generate_report()
```

This makes programs:

Easier to understand
Easier to test
Easier to debug
Easier to maintain
Reusable

3. Types of Functions in Python

There are two broad categories:

1. Built-in Functions

Python already provides them.

Examples:

print()

len()

sum()

max()

min()

type()

3DWEBSOFT FOUNDATION

input()

2. User-Defined Functions

Functions created by programmers.

Example:

```
def calculate_total():
    ...
```

4. Basic Function Syntax

```
def function_name():
    # function body
    statement
```

Example:

```
def greet():
    print("Hello, Student!")

greet()
```

Output:

```
Hello, Student!
```

Important Parts

```
def greet():
|   |
|   └── Function name
|
└── Keyword used to define a function
```

The parentheses () are followed by a colon :.

3DWEBSOFT FOUNDATION

5. Defining and Calling a Function

Step 1 – Define

```
def welcome():
    print("Welcome to Advanced Python!")
```

Step 2 – Call

```
welcome()
```

A function does not execute simply because we define it.

It executes when we **call** it.

Define Function

↓

Function Stored

↓

Call Function

↓

Function Executes

6. Function with Parameters

A **parameter** allows a function to receive information.

```
def greet(name):
    print("Hello", name)


greet("Aisha")
greet("Rahul")
```

Output:

```
Hello Aisha
```

3DWEBSOFT FOUNDATION

Hello Rahul

Here:

- name → parameter
- "Aisha" → argument

7. Parameter vs Argument

These two terms are often confused.

Parameter

Variable written in the function definition:

def greet(name):

name is the parameter.

Argument

Actual value passed to the function:

greet("Aisha")

"Aisha" is the argument.

```
def greet(name):
          ↑
      Parameter


greet("Aisha")
     ↑
  Argument
```

8. Multiple Parameters

A function can accept multiple parameters.

3DWEBSOFT FOUNDATION

```
def add(a, b):
    print(a + b)

add(10, 20)
```

Output:

30

Another example:

```
def student_info(name, age, grade):
    print("Name:", name)
    print("Age:", age)
    print("Grade:", grade)

student_info("Aisha", 17, "A")
```

9. Return Statement

A function can **return a result**.

Example:

```
def add(a, b):
    return a + b

result = add(10, 20)

print(result)
```

Output:

30

3DWEBSOFT FOUNDATION

print() vs return

print() displays something.

return sends a value back to the caller.

Example:

```
def add(a, b):
    return a + b
```

Now we can use the result:

```
total = add(10, 20)

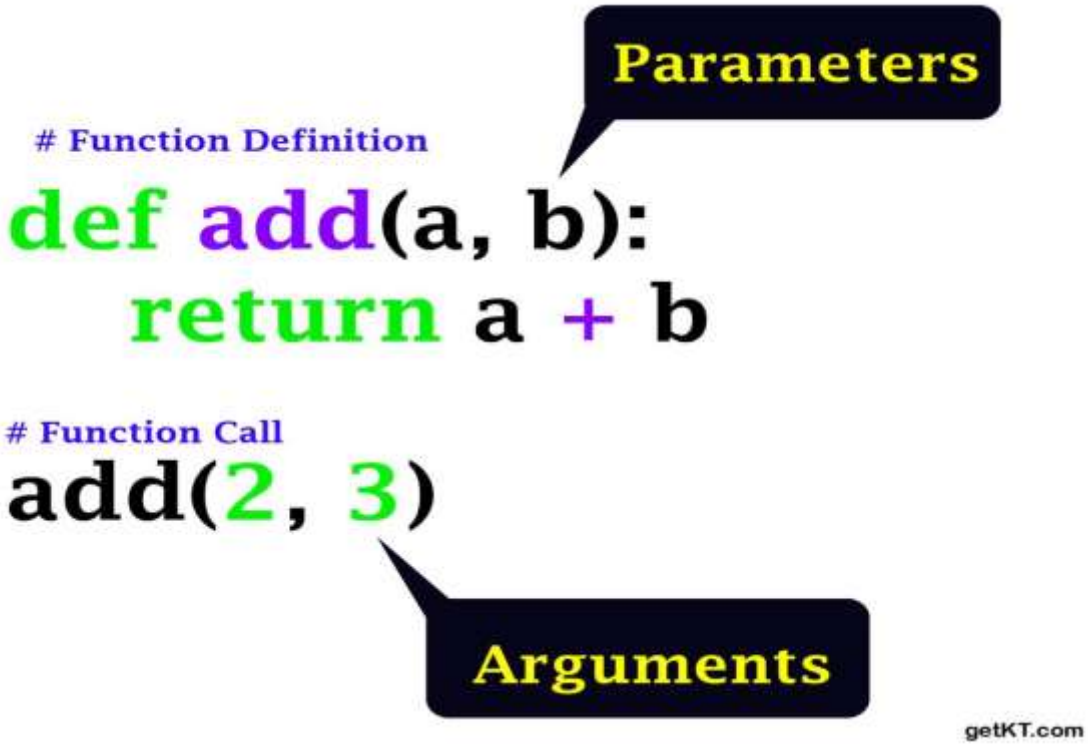

double = total * 2


print(double)
```

Output:

60

QFT FOUNDATION



10. Function with No Return

```
def greet(name):
    print("Hello", name)

result = greet("Aisha")

print(result)
```

Output:

Hello Aisha

None

If a function does not explicitly return a value, Python returns None.

11. Function with a Return Value

```
def square(number):
    return number ** 2

answer = square(5)

print(answer)
```

Output:

25

12. Returning Multiple Values

Python can return multiple values.

```
def calculate(a, b):
```

3DWEBSOFT FOUNDATION

```
    total = a + b
    difference = a - b
    product = a * b

    return total, difference, product

result = calculate(10, 5)

print(result)
```

Output:

(15, 5, 50)

We can unpack the values:

```
total, difference, product = calculate(10, 5)

print("Total:", total)
print("Difference:", difference)
print("Product:", product)
```

13. Default Arguments

A function can have a default value.

```
def greet(name="Student"):
    print("Hello", name)

greet()
greet("Aisha")
```

3DWEBSOFTFOUNDATION

Output:

Hello Student

Hello Aisha

If no argument is provided, "Student" is used.

14. Keyword Arguments

We can specify the parameter name when calling the function.

```
def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

student_info(age=17, name="Aisha")
```

Output:

Name: Aisha

Age: 17

The order does not matter when keyword arguments are used correctly.

15. Positional Arguments

Normally, arguments are matched according to their position.

```
def student(name, age):
    print(name)
    print(age)

student("Aisha", 17)
```

Here:

3DWEBSOFT FOUNDATION

name → "Aisha"

age → 17

Changing the order can change the meaning:

student(17, "Aisha")

So positional arguments must be supplied in the correct order.

16. Combining Positional and Default Arguments

```
def student(name, age=17):
    print("Name:", name)
    print("Age:", age)

student("Aisha")
student("Rahul", 18)
```

Output:

Name: Aisha

Age: 17

Name: Rahul

Age: 18

17. Variable Scope

Scope determines where a variable can be accessed.

There are two important concepts at this level:

Local Variable

Created inside a function.

3DWEBSOFT FOUNDATION

```
def calculate():
    total = 100
    print(total)

calculate()
```

total is local to the function.

Trying to use it outside the function:

```
print(total)
```

will cause an error because it is not defined in that outer scope.

18. Global Variable

A variable created outside a function is generally in the global scope.

```
school = "ABC School"

def display():
    print(school)

display()
```

The function can read the global variable.

Good Practice

Prefer passing values into functions instead of depending heavily on global variables.

For example, prefer:

```
def greet(school_name):
    print(school_name)
```

3DWEBSOFT FOUNDATION

rather than creating many global variables.

19. Functions and Lists

Functions can accept lists as arguments.

```
def calculate_total(marks):
    return sum(marks)

marks = [85, 90, 78, 92]

total = calculate_total(marks)

print("Total:", total)
```

Output:

Total: 345

This is very useful for data processing.

20. Function to Calculate Average

```
def calculate_average(marks):
    return sum(marks) / len(marks)

marks = [85, 90, 78, 92, 88]

average = calculate_average(marks)

print("Average:", average)
```

3DWEBSOFT FOUNDATION

Output:

Average: 86.6

21. Function to Find Grade

```
def calculate_grade(average):


    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    else:
        return "D"
```

Use it:

```
grade = calculate_grade(86.6)


print("Grade:", grade)
```

Output:

Grade: A

22. Reusable Functions

The biggest advantage of functions is **reusability**.

3DWEBSOFT FOUNDATION

Instead of writing:

total = sum(marks)

average = total / len(marks)

again and again, create:

```
def calculate_average(marks):
    return sum(marks) / len(marks)
```

Then use:

calculate_average(marks1)

calculate_average(marks2)

calculate_average(marks3)

One function can process many different datasets.

Practical Session

Practical 1 – Calculator Using Functions

```
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b


def multiply(a, b):
```

3DWEBSOFT FOUNDATION

```
    return a * b


def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b


print("Addition:", add(10, 5))
print("Subtraction:", subtract(10, 5))
print("Multiplication:", multiply(10, 5))
print("Division:", divide(10, 5))
```

Why is this better?

Instead of one large block, each function performs **one specific task**.

Practical 2 – Student Marks Functions

Create separate reusable functions:

```
def calculate_total(marks):
    return sum(marks)


def calculate_average(marks):
    return sum(marks) / len(marks)
```

3DWEBSOFT FOUNDATION

```
def highest_mark(marks):
    return max(marks)


def lowest_mark(marks):
    return min(marks)


marks = [85, 90, 78, 92, 88]

print("Total:", calculate_total(marks))
print("Average:", calculate_average(marks))
print("Highest:", highest_mark(marks))
print("Lowest:", lowest_mark(marks))
```

Practical 3 – Complete Student Result Analyzer

Now combine reusable functions.

```
def calculate_total(marks):
    return sum(marks)

def calculate_average(marks):
    return sum(marks) / len(marks)
```

3DWEBSOFT FOUNDATION

```
def calculate_grade(average):

    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    else:
        return "D"

def check_result(marks):
    for mark in marks:
        if mark < 40:
            return "FAIL"

    return "PASS"

student_name = input("Enter student name: ")

marks = []

for i in range(5):
```

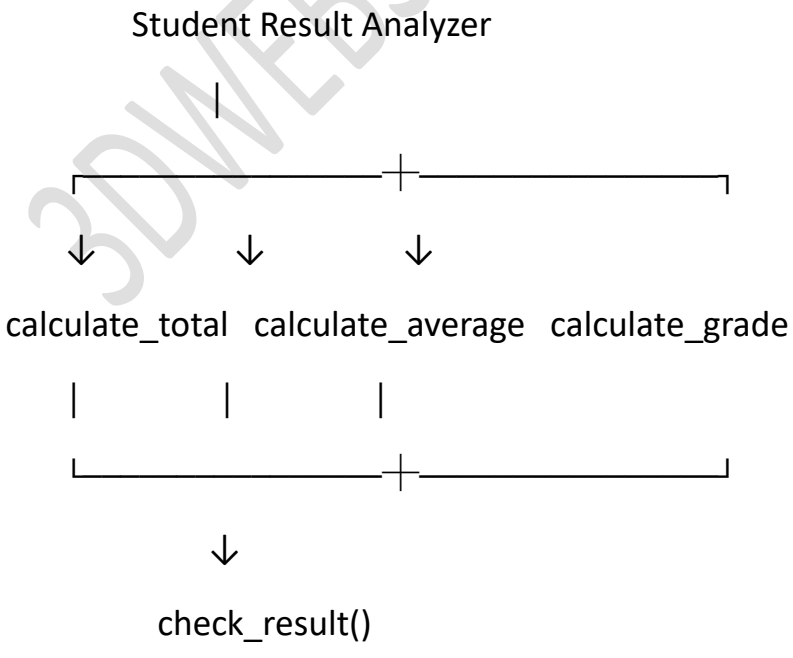
3DWEBSOFT FOUNDATION

```
    mark = int(input(f"Enter mark {i + 1}: "))
    marks.append(mark)

total = calculate_total(marks)
average = calculate_average(marks)
grade = calculate_grade(average)
result = check_result(marks)

print("\n===== STUDENT RESULT =====")
print("Name:", student_name)
print("Marks:", marks)
print("Total:", total)
print("Average:", average)
print("Grade:", grade)
print("Result:", result)
```

Program Structure



3DWEBSOFT FOUNDATION

↓

Final Report

Practical 4 – Function for Even Numbers

```
def find_even_numbers(numbers):
    return [number for number in numbers if number % 2 == 0]


numbers = [10, 15, 20, 25, 30, 35]

even_numbers = find_even_numbers(numbers)

print("Even Numbers:", even_numbers)
```

Output:

Even Numbers: [10, 20, 30]

This connects today's lesson with the previous class on **comprehensions**.

Practical 5 – Function for Duplicate Removal

```
def remove_duplicates(data):
    return list(set(data))


numbers = [10, 20, 10, 30, 20, 40]

unique_numbers = remove_duplicates(numbers)
```

3DWEBSOFT FOUNDATION

```
print("Original:", numbers)
print("Unique:", unique_numbers)
```

For cases where original order matters, use:

```
def remove_duplicates(data):
    return list(dict.fromkeys(data))
```

This preserves the first occurrence order.

Practical 6 – Temperature Converter

```
def celsius_to_fahrenheit(celsius):
    return (celsius * 9 / 5) + 32

print(celsius_to_fahrenheit(0))
print(celsius_to_fahrenheit(25))
print(celsius_to_fahrenheit(37))
```

Output:

```
32.0
77.0
98.6
```

The function can be reused for any Celsius value.

Practical 7 – Simple Interest Calculator

Formula:

SI = (P × R × T) / 100

Function:

```
def simple_interest(principal, rate, time):
```

3DWEBSOFT FOUNDATION

```
    return (principal * rate * time) / 100


result = simple_interest(10000, 5, 2)

print("Simple Interest:", result)
```

Output:

Simple Interest: 1000.0

Advanced Concept – Functions Calling Functions

One function can use another function.

```
def calculate_average(marks):
    return sum(marks) / len(marks)

def calculate_grade(marks):

    average = calculate_average(marks)

    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    else:
```

3DWEBSOFT FOUNDATION

```
    return "C"


marks = [85, 90, 88]

print(calculate_grade(marks))
```

This is useful for building larger applications from smaller reusable components.

Advanced Concept – Function as an Argument

Python allows functions to be passed to other functions.

Example:

```
def square(x):
    return x * x


def process(numbers, function):
    result = []

    for number in numbers:
        result.append(function(number))

    return result


numbers = [1, 2, 3, 4]
```

3DWEBSOFT FOUNDATION

print(process(numbers, square))

Output:

[1, 4, 9, 16]

Here, square is passed to process().

This is an important idea that prepares students for concepts such as **higher-order functions, map(), lambda functions, and functional programming**.

Practical Session

Practical Activity – "Function Builder"

Ask students to create separate functions for:

calculate_total()

calculate_average()

find_highest()

find_lowest()

calculate_grade()

check_result()

Then combine all functions into one program.

Input

Student: Aisha

Python: 92

Maths: 85

Physics: 78

English: 88

3DWEBSOFT FOUNDATION

Computer: 95

Expected Report

===== STUDENT REPORT =====

Name: Aisha

Marks: [92, 85, 78, 88, 95]

Total: 438

Average: 87.6

Highest: 95

Lowest: 78

Grade: A

Result: PASS

Debugging Activity

Find the problem:

```
def calculate_total(marks):
    sum(marks)

marks = [80, 90, 70]

total = calculate_total(marks)

print(total)
```

Problem

3DWEBSOFT FOUNDATION

The function calculates sum(marks) but does not return the result.

Correct Version

```
def calculate_total(marks):
    return sum(marks)
```

Common Mistakes

Mistake 1 – Forgetting parentheses when calling

Incorrect:

```
greet
```

Correct:

```
greet()
```

Mistake 2 – Forgetting return

Incorrect:

```
def add(a, b):
    a + b
```

Correct:

```
def add(a, b):
    return a + b
```

Mistake 3 – Wrong indentation

Incorrect:

```
def greet():
print("Hello")
```

Correct:

3DWEBSOFT FOUNDATION

```
def greet():
    print("Hello")
```

Mistake 4 – Using a local variable outside its function

```
def calculate():
    total = 100

print(total)
```

total is local to calculate().

Mistake 5 – Too many responsibilities in one function

Avoid creating functions such as:

```
def do_everything():
```

Instead, divide the work:

```
get_student()
calculate_total()
calculate_average()
calculate_grade()
display_report()
```

3DWEBSOFT FOUNDATION

Best Practices for Reusable Functions

1. Give functions meaningful names

Good:

```
calculate_average()
```

Poor:

x()

2. Keep one main responsibility

A function should ideally perform one clear task.

3. Use parameters

Make functions flexible.

```
def square(number):
    return number ** 2
```

4. Return results when appropriate

Returning makes a function useful in other parts of the program.

5. Avoid unnecessary global variables

Pass data through parameters.

6. Keep functions reasonably small

Smaller functions are generally easier to test and debug.

7. Use comments/docstrings when useful

```
def calculate_average(marks):
    """Return the average of a list of marks."""
    return sum(marks) / len(marks)
```

Real-World Applications

Reusable functions are used everywhere in software development.

School Software

↓

Functions

Banking Application

3DWEBSOFTFOUNDATION

↓

Functions

E-Commerce Website

↓

Functions

Data Science

↓

Functions

Machine Learning

↓

Functions

AI Applications

↓

Functions

Examples:

School System

calculate_marks()

calculate_grade()

Shopping System

calculate_total()

apply_discount()

3DWEBSOFT FOUNDATION

calculate_tax()

AI System

clean_data()

prepare_data()

predict_result()

Cybersecurity

check_password()

validate_input()

generate_report()

Interactive Challenge

Give students the following data:

marks = [78, 92, 65, 88, 95, 34]

Ask them to create reusable functions for:

Challenge 1

Find total.

Challenge 2

Find average.

Challenge 3

Find highest mark.

Challenge 4

Find lowest mark.

Challenge 5

Find all passing marks.

Challenge 6

3D WEBSOFT FOUNDATION

Find all failing marks.

Challenge 7

Calculate grade.

Challenge 8

Display a complete report.

Quiz

1. What is a function?

Answer: A reusable block of code designed to perform a specific task.

2. Which keyword creates a function?

Answer: def

3. How do we call a function?

Example:

greet()

4. What is a parameter?

Answer: A variable defined inside the function's parameter list.

5. What is an argument?

Answer: An actual value passed to a function.

6. Which keyword sends a result back?

Answer: return

7. What happens if a function has no explicit return?

Answer: It returns None.

8. What is a default argument?

Answer: A parameter that has a predefined value.

9. What is a local variable?

3DWEBSOFT FOUNDATION

Answer: A variable whose scope is limited to the function where it is defined.

10. Why are functions useful?

Answer: They make programs reusable, organized, easier to test, maintain, and debug.

Homework

Task 1 – Calculator Functions

Create:

add()

subtract()

multiply()

divide()

and build a calculator using them.

Task 2 – Student Functions

Create:

calculate_total()

calculate_average()

highest_mark()

lowest_mark()

calculate_grade()

Use them to generate a student report.

Task 3 – Number Analyzer

Create functions to:

3DWEBSOFT FOUNDATION

- Find even numbers
- Find odd numbers
- Find maximum
- Find minimum
- Calculate average

Task 4 – Real-Life Function

Choose one application and create at least **5 reusable functions**:

- Shopping bill
- Library management
- Student management
- Bank account
- Attendance system

2-Hour Class Plan

Time	Activity
0–10 min	Revision of comprehensions
10–25 min	What are functions?
25–40 min	Parameters and arguments
40–55 min	return, default & keyword arguments
55–65 min	Local and global scope
65–85 min	Practical reusable functions
85–105 min	Student Result Analyzer

3DWEBSOFT FOUNDATION

Time	Activity
105–112 min	Debugging challenge
112–117 min	Quiz
117–120 min	Homework & recap

Quick Reference

```
# Basic function
def greet():
    print("Hello")

# Function with parameter
def greet(name):
    print("Hello", name)

# Function with return
def add(a, b):
    return a + b

# Default argument
def greet(name="Student"):
    print("Hello", name)

# Multiple parameters
def student(name, age, grade):
```

3DWEBSOFT FOUNDATION

```
    print(name, age, grade)


# Function with a list
def calculate_total(marks):
    return sum(marks)
```

Key Takeaway

Students should remember:

FUNCTION

↓

Define once

↓

Give inputs through parameters

↓

Process the data

↓

Return the result

↓

Reuse whenever needed

Golden Rule

Write code once, organize it into a function, and reuse it whenever you need the same task.

3DWEBSOFT FOUNDATION